

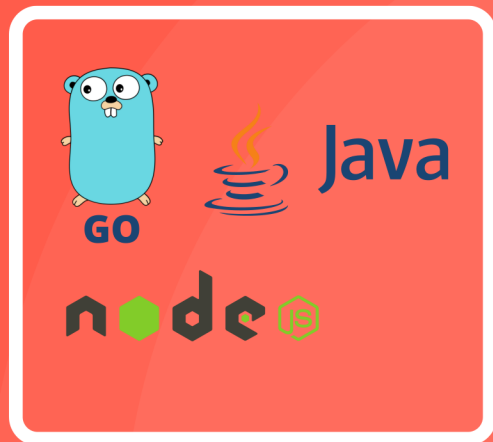
BACKEND MASTER CLASS #39

gRPC

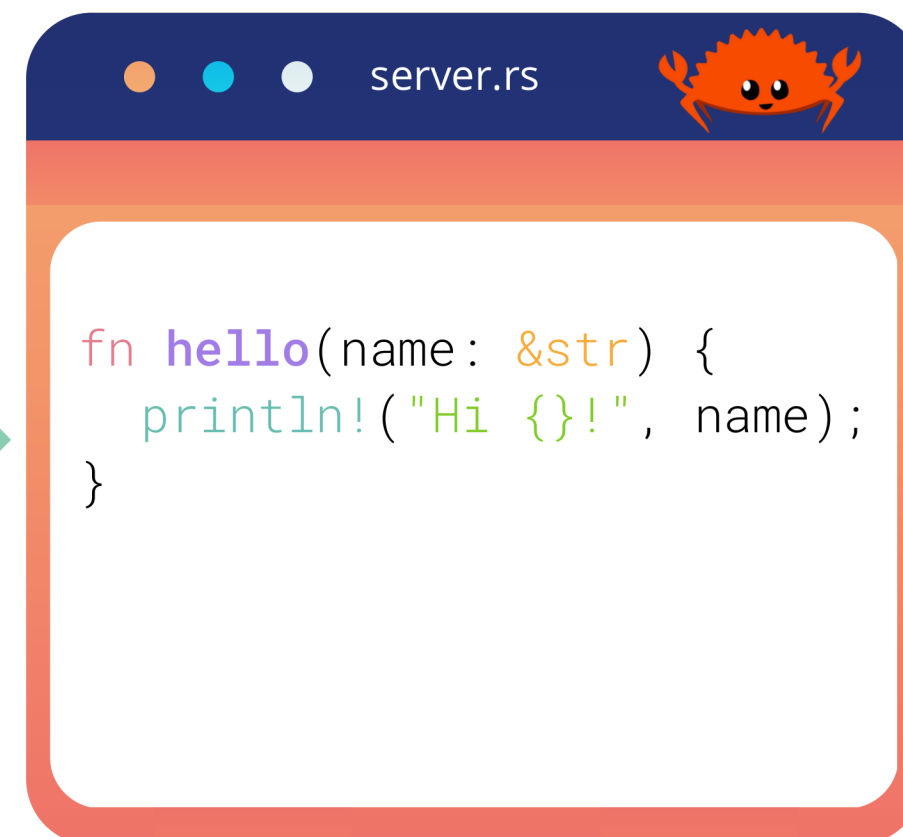




WHAT IS gRPC?



Calls over network



Remote Procedure Call Framework

- The client can execute a remote procedure on the server
- The remote interaction code is handled by gRPC
- The API & data structure code is automatically generated
- Support multiple programming languages

Originally developed by Google

- Now it is a part of the Cloud Native Computing Foundation CNCF

HOW IT WORKS?

1. Define API & data structure

The RPC and its request/response structure are defined using protobuf

2. Generate gRPC stubs

Generate codes for the server and client in the language of your choice

welcome.rs

```
...
pub struct HelloRequest {
    pub name: ::std::string::String,
}

pub struct HelloResponse {
    pub greet: ::std::string::String,
}

pub struct WelcomeServiceClient {
    client: ::grpcio::Client,
}

pub trait WelcomeService {
    fn hello(&mut self, req: HelloRequest,
            sink: ::grpcio::UnarySink<HelloResponse>);
}
...
```



generate

welcome.proto

```
syntax = "proto3";

message HelloRequest {
    string name = 1;
}

message HelloResponse {
    string greet = 1;
}

service WelcomeService {
    rpc Hello(HelloRequest) returns (HelloResponse);
}
```

generate

welcome.go

```
...
type HelloRequest struct {
    Name string
}

type HelloResponse struct {
    Greet string
}

type WelcomeServiceClient interface {
    Hello(*HelloRequest) (*HelloResponse, error)
}

type WelcomeServiceServer interface {
    Hello(*HelloRequest) (*HelloResponse, error)
}
...
```



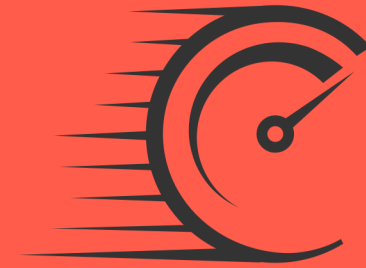
3. Implement the server

Implement the RPC handler on the server side

4. Use the client

Use the generated client stubs to call the RPC on the server

WHY GRPC?



High performance

HTTP/2: binary framing, multiplexing, header compression, bidirectional communication



Strong API contract

Server & client share the same protobuf RPC definition with strongly typed data

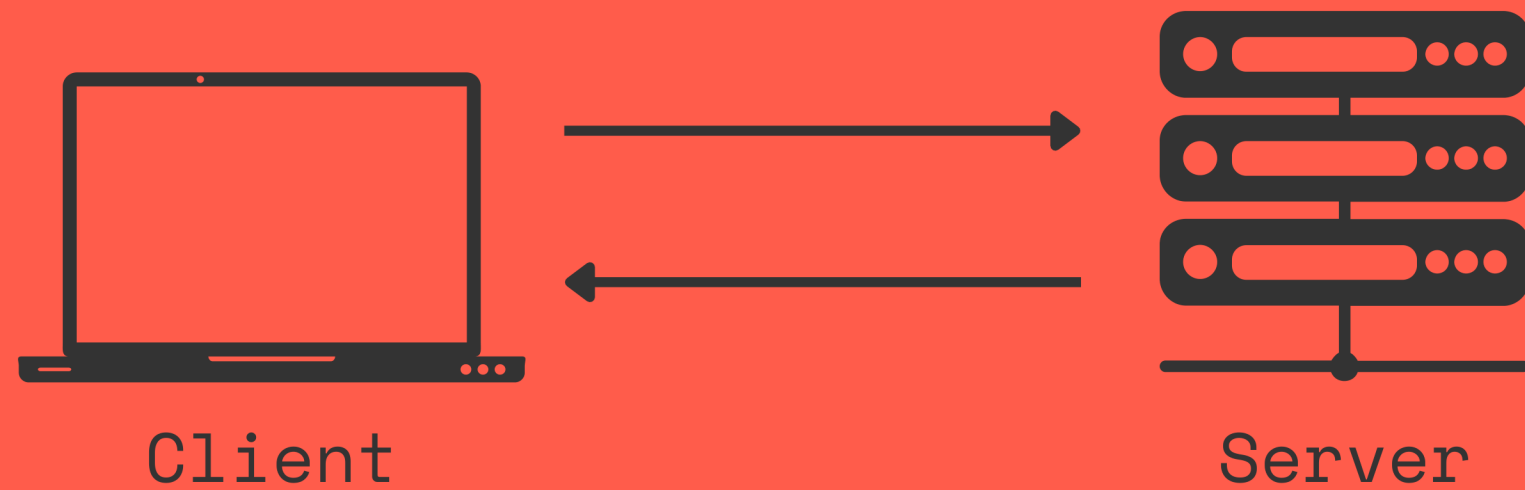


Automatic code generation

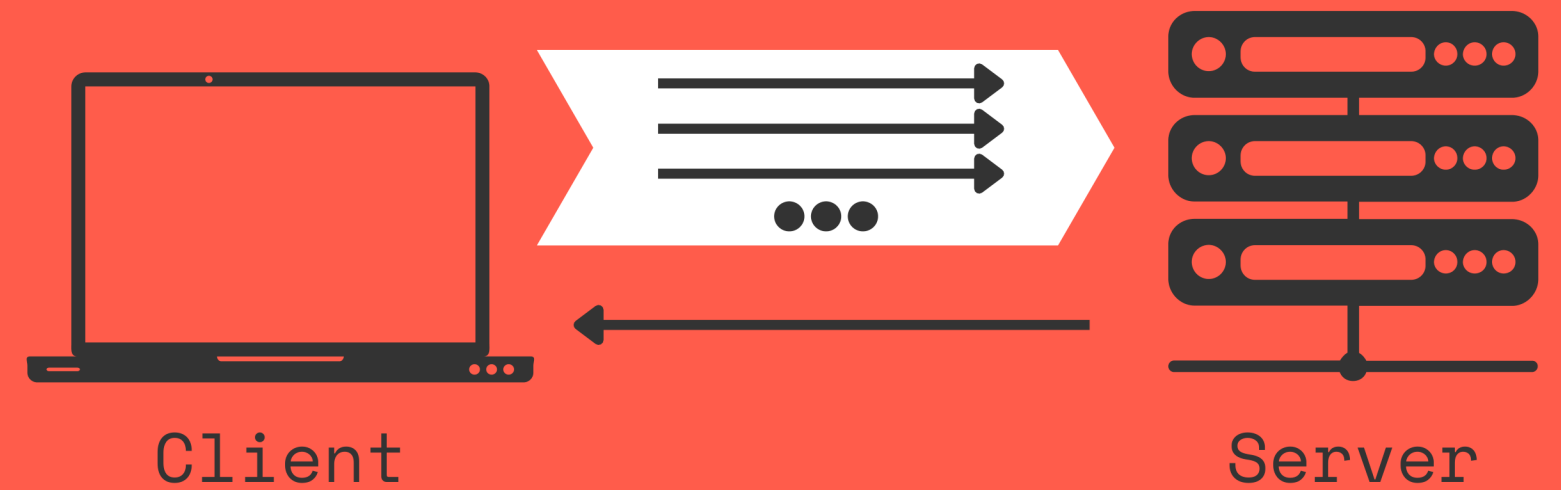
Codes that serialize/deserialize data, or transfer data between client & server are automatically generated

4 TYPES OF GRPC?

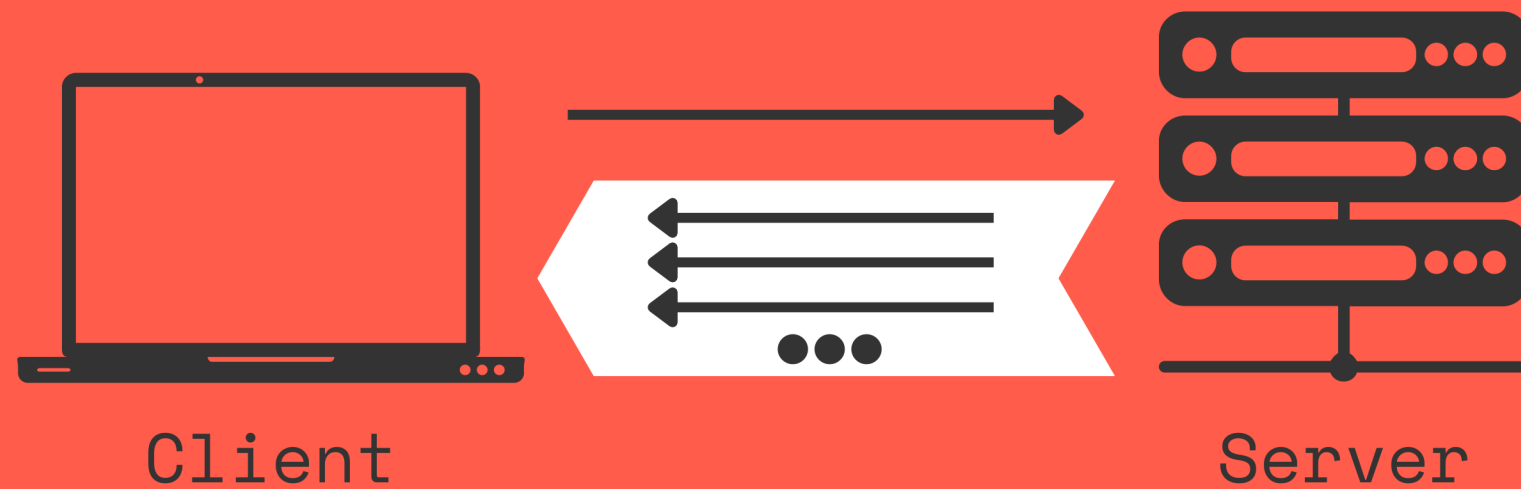
Unary gRPC



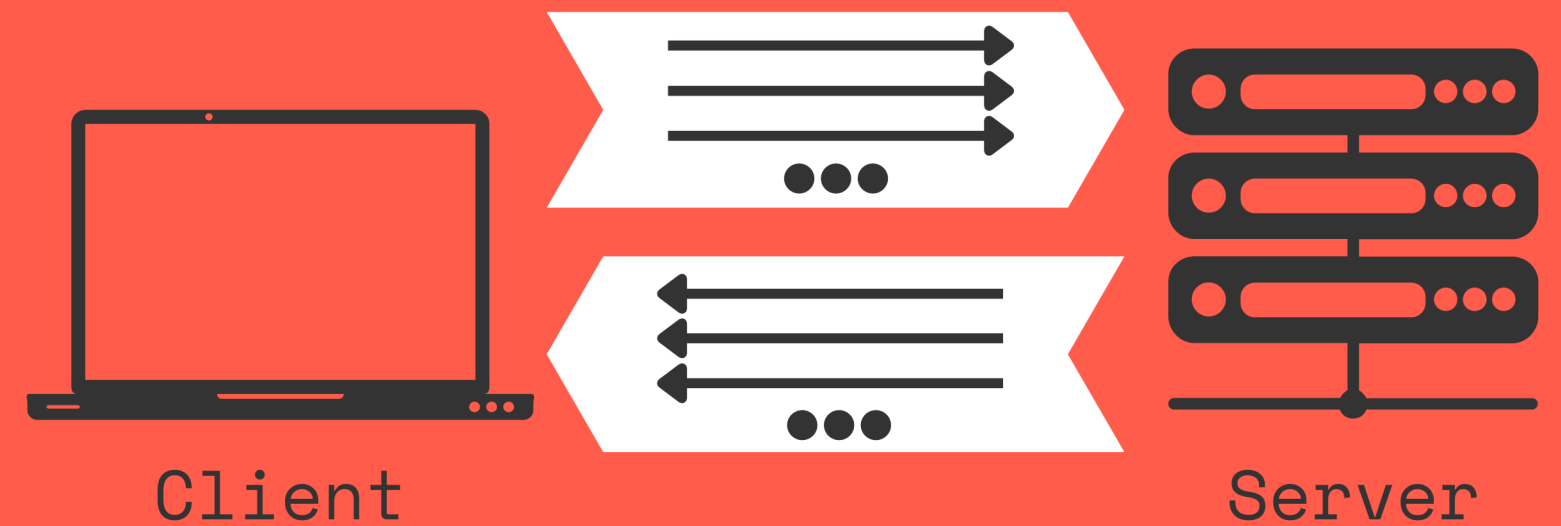
Client streaming gRPC



Server streaming gRPC



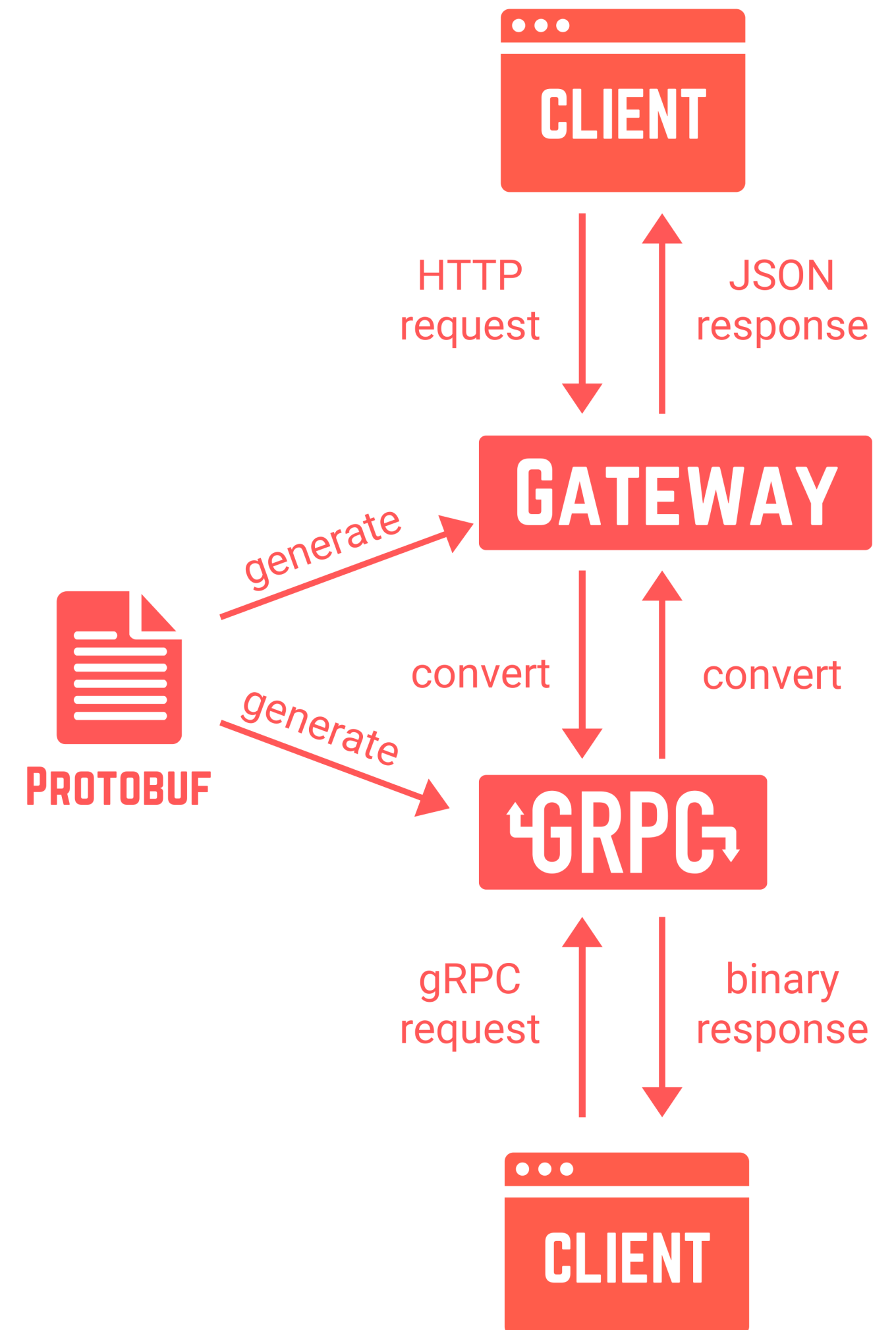
Bidirectional streaming gRPC



GRPC GATEWAY

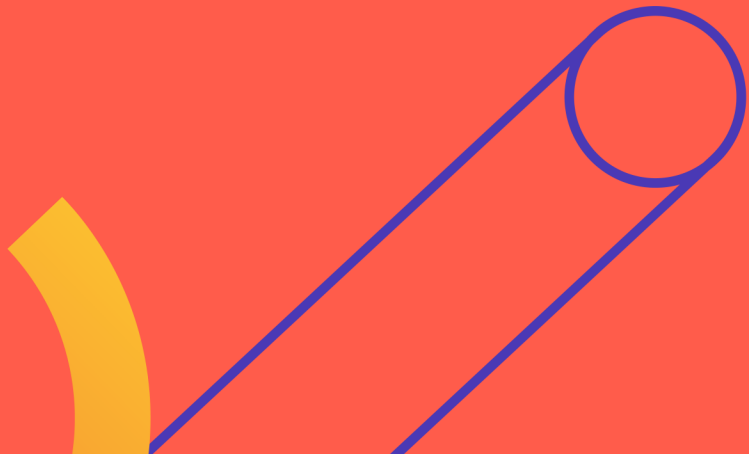
SERVE BOTH GRPC AND HTTP REQUESTS AT ONCE

- A plugin of protobuf compiler
- Generate proxy codes from protobuf
- Translate HTTP JSON calls to gRPC
 - In-process translation: only for unary
 - Separate proxy server: both unary and streaming
- Write code once, serve both gRPC and HTTP requests





WHAT'S NEXT?

- Set up gRPC development environment
 - Define the APIs & data structures using protobuf
 - Generate Golang codes from protobuf definition
 - Implement gRPC service handlers
 - Generate and run gRPC gateway to serve both gRPC & HTTP requests at once
 - Generate and embed a swagger documentation page to the server
- 
- 